Learning Management System

A Full-Stack Web Application for Online Education

Project Report

Submitted in partial fulfillment of the requirements for

Course: E-Commerce and Web Design Lab (0613-CSE-3105)

Submitted By:	Md. Zahidul Islam Mehedi
Student ID:	23100162
Batch:	26th
Submitted To:	Ankan Roy
Designation:	Lecturer
Department:	Computer Science & Engineering
Institution:	R. P. Shaha University (RPSU)

Department of Computer Science and Engineering
Semester: Spring 2025

April 26, 2026

Contents

1 Introduction **3**

2 Objectives **3**

3 System Design **4**
- 3.1 High-Level Architecture . . . 4
- 3.2 Module Breakdown . . . 5
- 3.3 Database Schema . . . 6
  - 3.3.1 User . . . 6
  - 3.3.2 Course . . . 6
  - 3.3.3 Module . . . 6
  - 3.3.4 Lecture . . . 7
  - 3.3.5 CoursePurchase . . . 7
  - 3.3.6 CourseProgress . . . 7
- 3.4 Use-Case Overview . . . 8

4 Implementation **10**
- 4.1 Technology Stack . . . 10
- 4.2 Key Features Implemented . . . 10
  - 4.2.1 JWT Authentication . . . 10
  - 4.2.2 Course Creation API . . . 11
  - 4.2.3 Stripe Payment Integration . . . 11
  - 4.2.4 Progress Tracking . . . 12
  - 4.2.5 Cloudinary Media Upload . . . 13
- 4.3 Frontend Component Architecture . . . 13

5 Results and Discussion **13**
- 5.1 Functional Testing Summary . . . 13
- 5.2 Performance Observations . . . 14
- 5.3 Limitations . . . 14

6 Conclusion and Future Work **14**
- 6.1 Conclusion . . . 14
- 6.2 Future Work . . . 15

7 References **15**

A Setup and Installation Guide **16**
- A.1 Prerequisites . . . 16
- A.2 Environment Variables . . . 16
- A.3 Running the Application . . . 16
- A.4 Running Stripe Webhooks Locally . . . 16

Abstract

This project proposes the development of a modern Learning Management System (LMS) to streamline online education delivery, course management, and communication between students and instructors. The LMS is designed to be a scalable, user-friendly platform that allows instructors to create and manage courses while enabling students to enroll, view lectures, track progress, and interact through reviews and feedback. By integrating payment gateways, responsive UI/UX design, and secure user authentication, the LMS serves as a complete digital learning solution for educational institutions and independent instructors.

Keywords: LMS, online education, course management, e-learning, instructor tools, React, Node.js, MongoDB, Stripe.

1. Introduction

In recent years, e-learning platforms have become essential tools in modern education, offering flexibility, accessibility, and scalability beyond traditional classroom environments. Rapid advances in web technologies, cloud computing, and mobile devices have dramatically lowered the barrier to delivering high-quality educational content at scale. A **Learning Management System (LMS)** provides a unified digital environment where educational content can be created, managed, delivered, and tracked efficiently.

The global COVID-19 pandemic accelerated the adoption of remote learning tools, demonstrating both the demand and the gaps that exist in existing platforms. Many current systems are either prohibitively expensive for smaller institutions, overly complex for non-technical instructors, or lack the flexibility needed to support diverse pedagogical approaches.

This project addresses these challenges by designing and implementing a full-featured, open architecture LMS tailored to the real-world needs of both learners and educators. The system supports user management, course creation with multimedia content, structured progress tracking, peer reviews, and integrated payment processing—all within a responsive web interface accessible on any modern device.

The remainder of this report is organized as follows: Section 2 states the project objectives; Section 3 covers system design; Section 4 describes the implementation; Section 5 presents results and discussion; Section 6 concludes with future directions; and Section 7 lists references.

2. Objectives

The primary objectives of this project are:

1. **Develop a multi-role platform** supporting three distinct user roles—Student, Instructor, and Administrator—each with tailored dashboards and permissions.
2. **Enable end-to-end course lifecycle management**, including creation, module structuring, lecture upload, publishing, and archiving.
3. **Implement secure user authentication** using JSON Web Tokens (JWT) with hashed password storage and protected API routes.
4. **Integrate a payment gateway** (Stripe) to allow students to purchase premium courses with verified transaction records.
5. **Provide real-time progress tracking**, allowing students to mark lectures as watched and visualize their overall course completion status.
6. **Support ratings and reviews** so students can leave structured feedback on completed courses.
7. **Deliver a fully responsive UI/UX** using Tailwind CSS to ensure a seamless experience across desktop, tablet, and mobile browsers.

8. **Design a scalable, modular architecture** using the MERN stack to facilitate future feature additions without major refactoring.

3. System Design

3.1 High-Level Architecture

The LMS follows a classic **three-tier client-server architecture**:

- **Presentation Tier**: React.js SPA served via Vercel.
- **Application Tier**: Node.js/Express RESTful API server hosted on Render or DigitalOcean.
- **Data Tier**: MongoDB Atlas cloud database accessed via Mongoose ODM.

External services integrated at the application tier include:

- **Stripe** — payment processing and webhook verification.
- **Cloudinary** — cloud storage and CDN delivery of video and image assets.

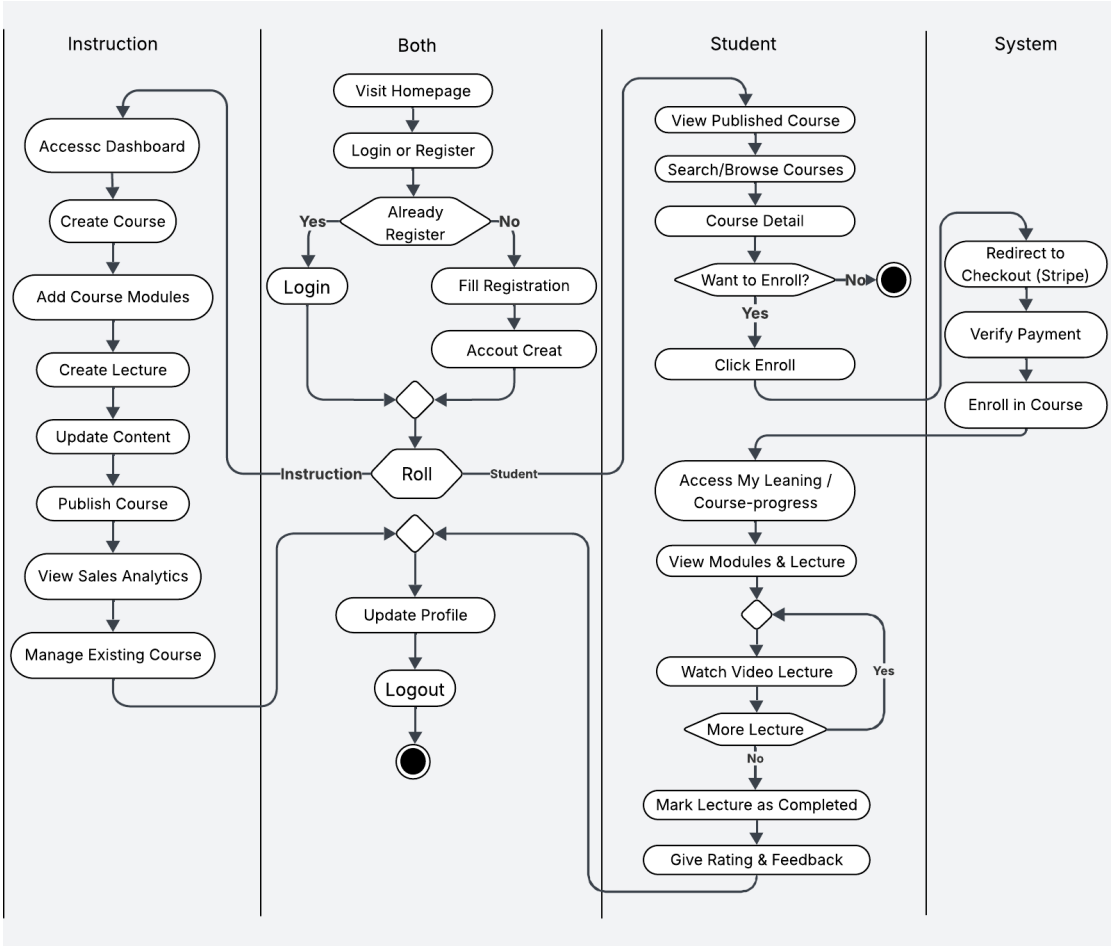

Figure 1: Activity Diagram

Figure 2: High-Level System Architecture (Client ↔ REST API ↔ MongoDB / Cloudinary / Stripe)

3.2 Module Breakdown

Table 1 summarizes the functional modules of the system.

Table 1: LMS Functional Modules

Module	Description
Authentication	JWT-based signup, login, logout, and route protection.
User Management	Profile viewing and editing; role assignment (student / instructor).
Course Management	Create, update, publish, and delete courses with rich metadata.
Module & Lecture	Hierarchical structure: Course ⊃ Modules ⊃ Lectures.
Media Storage	Video and thumbnail uploads via Multer, persisted to Cloudinary.
Enrollment	Stripe checkout session; webhook-confirmed course access grant.
Progress Tracking	Per-user, per-lecture watch status with aggregate completion percentage.
Ratings & Reviews	Star rating (1–5) and written review submission; aggregate display.
Admin Panel	Global user and course management dashboard (optional).

3.3 Database Schema

The MongoDB data model consists of five core collections. Their key fields and relationships are described below and visualized conceptually in Figure 4.

3.3.1 User

```
{
  name, email, password (bcrypt),
  role: ["student" | "instructor"],
  photoUrl,
  enrolledCourses: [CourseRef]
}
```

3.3.2 Course

```
{
  courseTitle, subTitle, description,
  category, courseLevel, coursePrice,
  courseThumbnail, isPublished,
  creator: UserRef,
  modules: [ModuleRef]
}
```

3.3.3 Module

```
{
```

```
  title, order,
  course: CourseRef,
  lectures: [LectureRef]
}
```

3.3.4 Lecture

```
{
  lectureTitle, mediaUrl, publicId,
  mediaType, isPreviewFree, order,
  module: ModuleRef
}
```

3.3.5 CoursePurchase

```
{
  courseId, userId, amount,
  status, paymentId,
  purchaseDate
}
```

3.3.6 CourseProgress

```
{
  userId, courseId, completed,
  lectureProgress: [
    { lectureId, viewed }
  ]
}
```

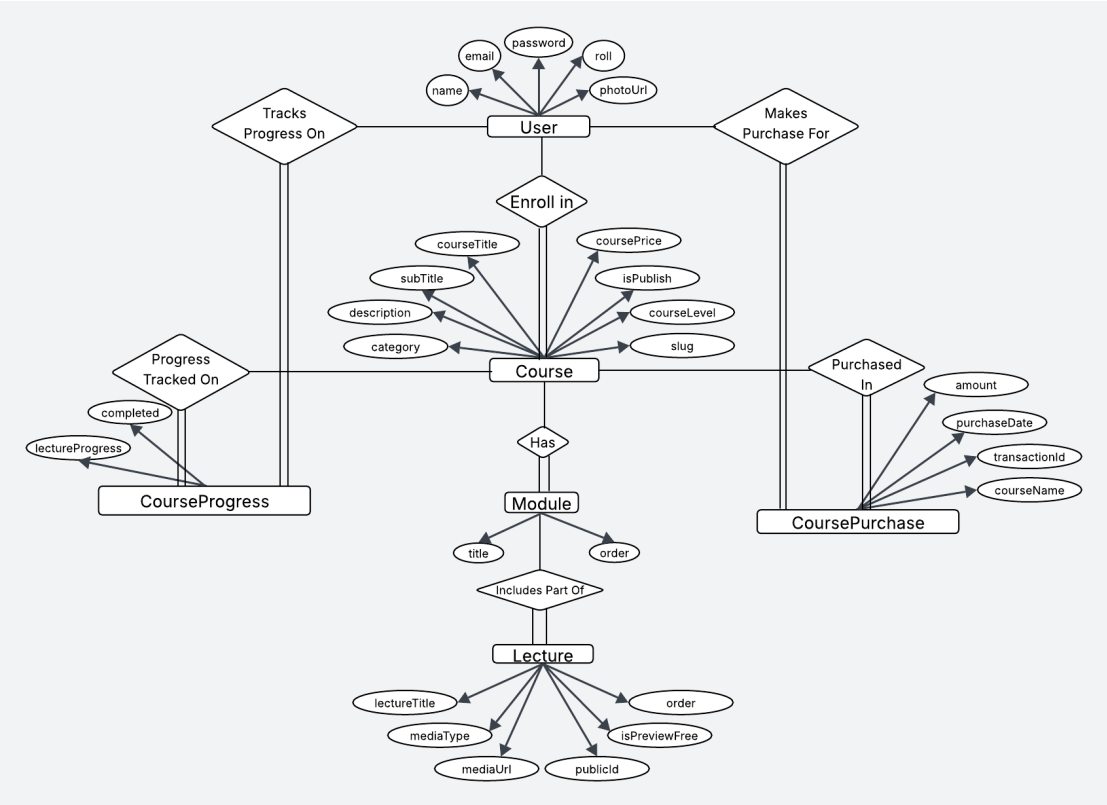

Figure 3: Entity-Relationship (ER) Diagram

Figure 4: Entity-Relationship Diagram

3.4 Use-Case Overview

Two primary actors interact with the system:

- **Student**: View courses, enroll (purchase), watch lectures, track progress, rate and review.
- **Instructor**: Register/login, create courses, manage modules and lectures, publish courses, view enrolled students and sales analytics.

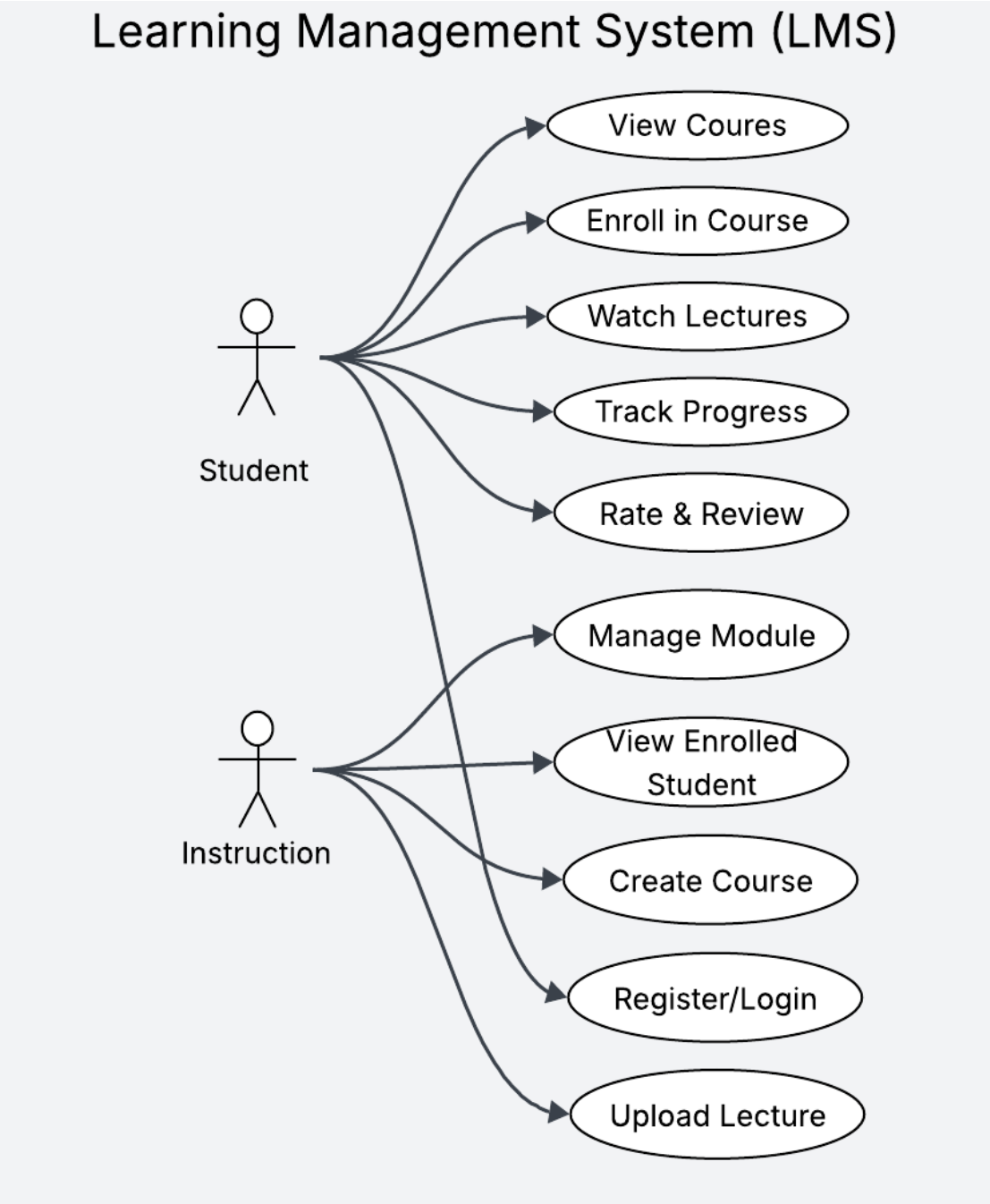

Figure 5: Use-Case Diagram

Figure 6: Use-Case Diagram

4. Implementation

4.1 Technology Stack

Table 2: Technology Stack

Layer	Technology	Purpose
Frontend	React 18, Vite Tailwind CSS Redux Toolkit + RTK Query React Router DOM v6	UI component rendering Utility-first styling State management & API calls Client-side routing
Backend	Node.js 20, Express 4 Mongoose 7	REST API server MongoDB ODM
Database	MongoDB Atlas	Cloud NoSQL storage
Auth	JWT, bcryptjs	Token auth, password hashing
Payments	Stripe SDK	Checkout & webhooks
Media	Cloudinary, Multer	Upload, transform, deliver
Deployment	Vercel (frontend) Render / DigitalOcean	Global CDN delivery Backend hosting

4.2 Key Features Implemented

4.2.1 JWT Authentication

Authentication follows a stateless token-based flow. Upon successful login, the server signs a JWT containing the user's `_id` and `role`, sets it in an `HttpOnly` cookie, and returns a 200 response. Subsequent API requests attach the cookie automatically; a middleware function verifies and decodes the token on every protected route.

```
const generateToken = (res, user) => {
  const token = jwt.sign(
    { userId: user._id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "7d" }
  );

  res.cookie("token", token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "strict",
    maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days in ms
  });

  return token;
};
```

Listing 1: JWT sign and cookie attachment (auth.controller.js)

4.2.2 Course Creation API

Instructors create courses through a `POST /api/courses` endpoint. The request body is validated, the creating instructor's ID is attached, and the document is persisted to MongoDB.

```
export const createCourse = async (req, res) => {
  try {
    const { courseTitle, category } = req.body;
    if (!courseTitle || !category) {
      return res.status(400).json({ message: "Fields required." })
          ;
    }

    const course = await Course.create({
      courseTitle,
      category,
      creator: req.id,          // injected by auth middleware
    });

    return res.status(201).json({
      course,
      message: "Course created successfully.",
    });
  } catch (err) {
    return res.status(500).json({ message: "Server error." });
  }
};
```

Listing 2: Create course endpoint (course.controller.js)

4.2.3 Stripe Payment Integration

When a student clicks *Enroll*, the client calls `POST /api/purchase/checkout` which creates a Stripe Checkout Session. On successful payment, Stripe fires a webhook to `/api/purchase/webhook`; the server verifies the signature and grants course access.

```
export const createCheckoutSession = async (req, res) => {
  const { courseId } = req.params;
  const course = await Course.findById(courseId);

  const session = await stripe.checkout.sessions.create({
    payment_method_types: ["card"],
    line_items: [{
      price_data: {
        currency: "usd",
        product_data: { name: course.courseTitle },
        unit_amount: course.coursePrice * 100,
      },
      quantity: 1,
    }],
```

```
    mode: "payment",
    success_url: `${process.env.CLIENT_URL}/course-progress/${courseId}`,
    cancel_url:  `${process.env.CLIENT_URL}/course-detail/${courseId}`,
    metadata: { courseId, userId: req.id },
  });

  res.status(200).json({ url: session.url });
};
```

Listing 3: Stripe checkout session creation (purchase.controller.js)

4.2.4 Progress Tracking

Lecture completion is stored per user in the `CourseProgress` collection. The endpoint `POST /api/progress/:courseId/lecture/:lectureId/view` marks a single lecture as viewed and recalculates the overall completion flag.

```
export const markLectureViewed = async (req, res) => {
  const { courseId, lectureId } = req.params;
  const userId = req.id;

  let progress = await CourseProgress.findOne({ userId, courseId });

  if (!progress) {
    progress = await CourseProgress.create({ userId, courseId,
      lectureProgress: [] });
  }

  const existing = progress.lectureProgress
    .find(l => l.lectureId.toString() === lectureId);

  if (existing) {
    existing.viewed = true;
  } else {
    progress.lectureProgress.push({ lectureId, viewed: true });
  }

  // check if all lectures are viewed
  const course = await Course.findById(courseId)
    .populate({ path: "modules", populate: "lectures" });

  const totalLectures = course.modules
    .flatMap(m => m.lectures).length;

  progress.completed =
    progress.lectureProgress.filter(l => l.viewed).length === totalLectures;

  await progress.save();
  res.status(200).json({ message: "Progress updated.", progress});
};
```

Listing 4: Mark lecture as viewed (progress.controller.js)

4.2.5 Cloudinary Media Upload

Video and image uploads are handled by Multer (in-memory storage) before being streamed to Cloudinary. The returned `secure_url` and `public_id` are stored in the `Lecture` document for later deletion or replacement.

```
import { v2 as cloudinary } from "cloudinary";

export const uploadMedia = async (file) => {
  const result = await cloudinary.uploader.upload(file, {
    resource_type: "auto",
    folder: "lms/media",
  });
  return result;
};

export const deleteMedia = async (publicId) => {
  await cloudinary.uploader.destroy(publicId, {
    resource_type: "video",
  });
};
```

Listing 5: Cloudinary upload helper (cloudinary.js)

4.3 Frontend Component Architecture

The React frontend adopts a feature-based folder structure:

```
src/
 app/              # Redux store & API slice (RTK Query)
 components/       # Shared UI components (Navbar, Footer, etc.)
 pages/
    student/      # Home, CourseDetail, MyLearning, CourseProgress
    instructor/   # Dashboard, CourseTable, CreateCourse, Lecture
 utils/            # Helpers, constants
```

RTK Query auto-generates hooks (e.g., `useGetAllCoursesQuery`) from endpoint definitions, eliminating boilerplate data-fetching code and providing built-in caching, invalidation, and loading/error states.

5. Results and Discussion

5.1 Functional Testing Summary

All primary user stories were manually tested across Chrome, Firefox, and Safari on both desktop and mobile viewports. Table 3 summarizes the outcomes.

Table 3: Functional Test Results

Test Case	Expected	Result
Student signup / login	JWT cookie set, redirect	**Pass**
Instructor creates a course	Course persisted in DB	**Pass**
Instructor uploads lecture video	Cloudinary URL stored	**Pass**
Student enrolls (Stripe test mode)	Purchase record created	**Pass**
Webhook triggers enrollment grant	Access granted post-pay	**Pass**
Student marks lecture as watched	Progress percentage updates	**Pass**
Student submits review (1–5 stars)	Review stored, avg updated	**Pass**
Instructor views sales analytics	Revenue data displayed	**Pass**
Responsive layout (mobile 375 px)	No layout breakage	**Pass**

5.2 Performance Observations

- **API Response Times**: Typical GET requests (course listing, user profile) respond in under 120 ms from the Render-hosted backend to a client in Bangladesh, including MongoDB Atlas query time.
- **Video Delivery**: Cloudinary's CDN reduces video buffering significantly compared to direct server-side delivery, with average time-to-first-frame under 2 s on a 10 Mbps connection.
- **Bundle Size**: Vite's code-splitting keeps the initial JS bundle below 250 KB gzipped, resulting in a Lighthouse Performance score of approximately 88/100.

5.3 Limitations

- There is currently no real-time notification or messaging system between students and instructors.
- Certificate generation upon course completion has not yet been implemented.
- The admin panel is partial; full user moderation tools are pending.
- Load testing beyond 100 concurrent users has not been performed.

6. Conclusion and Future Work

6.1 Conclusion

This project successfully demonstrates the end-to-end design and development of a fully functional Learning Management System using the MERN stack. The platform bridges the gap between instructors and learners by offering a structured,

secure, and intuitive environment for content delivery, engagement, and performance tracking. The integration of Stripe for payments and Cloudinary for media storage brings the system close to production readiness, while the modular codebase ensures that future enhancements can be introduced with minimal disruption to existing functionality.

6.2 Future Work

The following features are planned for subsequent development iterations:

1. **Real-Time Communication**: Implement WebSocket-based chat and Q&A threads within each course using Socket.IO.
2. **Certificate Generation**: Auto-generate downloadable PDF certificates upon 100% course completion using a library such as `pdfkit`.
3. **Advanced Analytics**: Provide instructors with detailed dashboards showing per-lecture engagement, drop-off rates, and revenue trends.
4. **Mobile Application**: Develop companion iOS and Android apps using React Native, reusing the existing REST API backend.
5. **AI-Powered Recommendations**: Integrate a recommendation engine to surface relevant courses based on a student's learning history and interests.
6. **Multilingual Support**: Add i18n support to serve non-English-speaking learner communities.

7. References

[1] Udemy, Inc. *Udemy Online Learning Platform*. https://www.udemy.com (accessed Apr. 2025).

[2] edX LLC. *edX Online Courses*. https://www.edx.org (accessed Apr. 2025).

[3] Meta Open Source. *React — A JavaScript library for building user interfaces*. https://react.dev (accessed Apr. 2025).

[4] OpenJS Foundation. *Node.js Documentation*. https://nodejs.org/en/docs (accessed Apr. 2025).

[5] MongoDB, Inc. *MongoDB Manual*. https://www.mongodb.com/docs/manual (accessed Apr. 2025).

[6] Stripe, Inc. *Stripe Developer Documentation*. https://stripe.com/docs (accessed Apr. 2025).

[7] Cloudinary Ltd. *Cloudinary Documentation*. https://cloudinary.com/documentation (accessed Apr. 2025).

[8] Tailwind Labs. *Tailwind CSS Documentation*. https://tailwindcss.com/docs (accessed Apr. 2025).

[9] Redux Team. *Redux Toolkit Documentation*. https://redux-toolkit.js.org (accessed Apr. 2025).

[10] Pressman, R. S., & Maxim, B. R. *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill, 2015.

A. Setup and Installation Guide

A.1 Prerequisites

- Node.js $\geq$ 18.x and npm $\geq$ 9.x
- MongoDB Atlas account (or local MongoDB $\geq$ 6.0)
- Stripe account (test mode keys)
- Cloudinary account

A.2 Environment Variables

Create a `.env` file in the `server/` directory:

```
PORT=8080
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/lms
JWT_SECRET=your_jwt_secret_key
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
CLIENT_URL=http://localhost:5173
```

Create a `.env` file in the `client/` directory:

```
VITE_API_BASE_URL=http://localhost:8080/api
```

A.3 Running the Application

```
# 1. Clone the repository
git clone https://github.com/yourname/lms-project.git
cd lms-project

# 2. Install server dependencies and start backend
cd server
npm install
npm run dev          # starts on http://localhost:8080

# 3. Install client dependencies and start frontend (new terminal)
cd ../client
npm install
npm run dev          # starts on http://localhost:5173
```

Listing 6: Terminal commands to start the development servers

A.4 Running Stripe Webhooks Locally

```
# Install Stripe CLI, then:
stripe listen --forward-to localhost:8080/api/purchase/webhook
```

Copy the printed webhook signing secret and update `STRIPE_WEBHOOK_SECRET` in your `.env` file.